

AutoMapper Conditional Mapping in C# with Examples

In this article, I am going to discuss **AutoMapper Conditional Mapping in C#** with Examples. Please read our previous article where we discussed [AutoMapper Reverse Mapping in C#](#) with Examples. At the end of this article, you will understand what is AutoMapper Conditional Mapping and when and how to use Conditional Mapping in AutoMapper. AutoMapper Conditional Mapping in C# is one of the important concepts of AutoMapper which is used in most real-time projects.

What is AutoMapper Conditional Mapping?

The **AutoMapper in C#** allows us to add conditions to the properties of the source object that must be met before that property going to be mapped to the property of the destination object. For example, if we want to map a property of the Source Object only if its value is greater than 0, then in such a situation we need to use AutoMapper Conditional Mapping.

Example to understand AutoMapper Conditional Mapping in C#

Let us understand AutoMapper Conditional Mapping in C# with an example. We are going to use the following two classes to Understand AutoMapper Conditional Mapping.

```
public class Product
{
    public int ProductID { get; set; }
    public string Name { get; set; }
    public string OptionalName { get; set; }
    public int Quantity { get; set; }
    public int Amount { get; set; }
}
```

```
public class ProductDTO
{
    public int ProductID { get; set; }
    public string ItemName { get; set; }
    public int ItemQuantity { get; set; }
    public int Amount { get; set; }
}
```

Create a class file with the name Product.cs and then copy and paste the following code into it. This is a very simple class having 5 primitive-type properties. This is going to be our Source Object.

```
namespace AutoMapperDemo
{
    public class Product
    {
        public int ProductID { get; set; }
        public string Name { get; set; }
        public string OptionalName { get; set; }
        public int Quantity { get; set; }
        public int Amount { get; set; }
    }
}
```

Next, create another class file with the name ProductDTO.cs and then copy and paste the following code into it. This is also a very simple class having 4 primitive-type properties. This is going to be our Destination Object.

```
namespace AutoMapperDemo
```

```

{
    public class ProductDTO
    {
        public int ProductID { get; set; }
        public string ItemName { get; set; }
        public int ItemQuantity { get; set; }
        public int Amount { get; set; }
    }
}

```

Business Requirement:

1. We need to Map the **Name** property of the **Product** class to the **ItemName** property of the **ProductDTO** class only if the Name value starts with the letter **A**, else Map the **OptionalName** property value of the **Product** class with the **ItemName** property of the **ProductDTO** class.
2. If the **Quantity** value of the Product class is greater than **0** then only map it to the **ItemQuantity** property of the ProductDTO class.
3. Similarly, if the **Amount** value of the Product class is greater than 100 then only map it to the **Amount** property of the ProductDTO class.

To achieve this we need to use AutoMapper Conditional Mapping in C#. So, create a class file with the name MapperConfig.cs and copy and paste the following code into it. Here, you can see. for the ItemName destination property, we are using ternary operator and setting the value. If the Name starts with A, then we are assigning the Name property value else we are assigning the OptionalName property value to the Destination Name property. Then using the AutoMapper Condition method, we are checking the value of the Property of the Source Object and if it satisfies the condition, then we are mapping the value to the destination property. If the condition failed, then it will store the default value based on the data type.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg =>
            {
                cfg.CreateMap<Product, ProductDTO>()
                    //If the Name Start with A then Map the Name Value else Map
OptionalName value
                    .ForMember(dest => dest.ItemName, act => act.MapFrom(src =>
                        (src.Name.StartsWith("A") ? src.Name : src.OptionalName)
                    //Map the quantity value if its greater than 0
                    .ForMember(dest => dest.ItemQuantity, act => act.Condition
(src.Quantity > 0)))
                    //Map the amount value if its greater than 100
                    .ForMember(dest => dest.Amount, act => act.Condition(src =>
(src.Amount > 100))));
            });
        }
    }
}

```

```

        //Creating the Mapper Instance
        var mapper = new Mapper(config);
        //returning the Mapper Instance
        return mapper;
    }
}
}

```

Note: The AutoMapper Condition method is used to add conditions to the properties that must be met before that property is going to map. The **MapFrom** option is used to perform the custom source and destination member mappings.

Next, modify the Main method of the Program class as follows to test whether the AutoMapper Conditional Mapping is working or not. Here, you can see, we are adding the Product Name as Led TV which does not start with A, and hence it should Map the OptionalName property value. Further, we have set the Quantity as -5 which is not greater than 0 and in this case, it will not Map the property value. In this case, it will take the default value i.e. 0 as the data type is Integer. Finally, we have set the Amount value as 1000 and this is greater than 0, which means this value will be mapped to the destination property.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var mapper = MapperConfig.InitializeAutomapper();
            Product product = new Product()
            {
                ProductID = 101,
                Name = "Led TV",
                OptionalName = "Product name not start with A",
                Quantity = -5,
                Amount = 1000
            };
            var productDTO = mapper.Map<Product, ProductDTO>(product);
            Console.WriteLine("Before Mapping : Product Object");
            Console.WriteLine("ProductID : " + product.ProductID);
            Console.WriteLine("Name : " + product.Name);
            Console.WriteLine("OptionalName : " + product.OptionalName);
            Console.WriteLine("Quantity : " + product.Quantity);
            Console.WriteLine("Amount : " + product.Amount);
            Console.WriteLine();
            Console.WriteLine("After Mapping : ProductDTO Object");
            Console.WriteLine("ProductID : " + productDTO.ProductID);
            Console.WriteLine("ItemName : " + productDTO.ItemName);
            Console.WriteLine("ItemQuantity : " + productDTO.ItemQuantity);
            Console.WriteLine("Amount : " + productDTO.Amount);
            Console.ReadLine();
        }
    }
}

```

```
}
```

Output:

```
Before Mapping : Product Object
ProductID : 101
Name : Led TV
OptionalName : Product name not start with A
Quantity : -5
Amount : 1000

After Mapping : ProductDTO Object
ProductID : 101
ItemName : Product name not start with A
ItemQuantity : 0
Amount : 1000
```

In the next article, I am going to discuss [how to use AutoMapper Ignore Property in C#](#) with Examples. Here, in this article, I try to explain **AutoMapper Conditional Mapping in C#** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.